

MindTrack-CLI

Design & Documentation Artefacts

VITyarthi - Build Your Own Project | Section 4

Project title	MindTrack-CLI: an offline command-line journaling, mood-tracking and stress self-check tool
Student name	Aman Kumar
Registration number	26BCE10427
Course / subject	CSE CORE and CSE1021
Language / tools	Python 3.13, standard library only (csv, datetime, random); local .txt and .csv storage
Document scope	Design artefacts required by Section 4 of the VITyarthi guidelines

Contents

1. Problem Statement
2. Objectives
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture Diagram
6. Process Flow / Workflow Diagram
7. UML Diagrams (Use Case, Class/Component, Sequence)
8. Database / Storage Design (ER Diagram and Schema)
9. ML-specific artefacts
10. Checklist against Section 4

4.1 Problem Statement

Awareness of mental health has grown, yet many people, and students in particular, still lack a consistent habit of emotional reflection. Three obstacles keep recurring: the lack of a **simple** tool, **privacy** worries about storing personal thoughts on online services, and apps that are **over-complicated** for a person who only wants a few quiet minutes of reflection.

MindTrack-CLI addresses this by providing an **offline, beginner-friendly, command-line application** that encourages self-reflection through journaling, daily mood logs and a short guided stress questionnaire, while keeping all data **locally** on the user's own device.

Target users

- Students who feel stressed by academic pressure and personal-life concerns.
- People with privacy concerns who prefer to keep their thoughts stored locally.

Important: MindTrack-CLI is a self-reflection aid. The stress questionnaire is a simple, non-clinical check and is not a diagnostic or medical tool.

4.2 Objectives

1. Provide a simple, menu-driven terminal interface that a beginner can operate without training.
2. Let users record free-text **journal entries** that are automatically dated and stored locally.
3. Let users log a **daily mood** and review their mood history.
4. Offer a short **stress questionnaire** that turns five answers into a Low / Moderate / High indication with supportive advice.
5. Offer **positive affirmations** on demand to encourage a healthy mindset.
6. Guarantee **privacy and offline use** by storing all data in local .txt and .csv files with no network access and no external packages.
7. Apply core Python concepts (functions, file I/O, the csv module, exceptions, loops and selection) in a modular, readable program.

4.3 Functional Requirements

The system consists of four major functional modules, Journal, Mood Tracker, Affirmation and Stress Assessment, coordinated by a Main Menu. Each requirement has a clear input and output, and the workflow is shown in Section 4.6.

ID	Module	Requirement	Input	Output
FR1	Main Menu	Show a numbered menu (options 1-7) in a loop until the user chooses Exit; reject any other input with an error message.	Menu choice (text)	Menu, or 'Invalid option. Try again.'
FR2	Journal	Accept one journal entry and append it to journal.txt as a line stamped with today's date. The file is created if it does not exist.	Free text entry	'Entry saved' confirmation; new line in journal.txt
FR3	Journal	Display all saved journal entries; show a friendly message when no journal file exists yet.	None	All entries, or 'No entries found yet.'
FR4	Mood Tracker	Ask how the user feels (happy / sad / anxious / neutral / angry), convert the answer to lower case and append [date, mood] to mood.csv.	Mood word	'Mood saved' confirmation; new row in mood.csv
FR5	Mood Tracker	Display every logged mood as 'date → mood'; show a friendly message when no mood file exists yet.	None	Mood log, or 'No mood entries found.'
FR6	Affirmation	Show one randomly chosen positive affirmation from a built-in list of five.	None	One affirmation
FR7	Stress Assessment	Ask five questions on a 1-5 scale, add the answers (range 5-25) and classify: up to 10 Low, 11-18 Moderate, above 18 High, with a short advice message.	Five integers (1-5)	Stress level and advice
FR8	Storage	Persist all data in local plain-text (.txt) and CSV files so that it is available in later sessions.	n/a	journal.txt, mood.csv
FR9	Main Menu	Exit cleanly with a goodbye message when option 7 is chosen.	Option 7	Goodbye message; program ends

4.4 Non-functional Requirements

Eight non-functional requirements are specified (the guideline asks for at least four). The status column reflects the current implementation honestly, so that partly met items can be planned as enhancements.

ID	Category	Requirement	How it is addressed in MindTrack-CLI	Status
NFR1	Usability	A first-time user must be able to use every feature without instructions beyond the on-screen prompts.	Numbered menu, plain-language prompts, confirmation messages (✓ Entry saved!), emoji-based result cues.	Met
NFR2	Privacy & Security	User data must never leave the user's device.	No network access and no third-party libraries; data is written only to local files. Files are plain text (not encrypted), so protection relies on the user's own device security.	Met
NFR3	Portability	Run on common operating systems with no installation of extra packages.	Uses only the Python standard library (csv, datetime, random). Developed for Python 3.13; verified during documentation testing on Python 3.12 / Linux.	Met
NFR4	Performance & Resource Efficiency	Respond instantly and use negligible memory and disk.	Appending a record is a constant-time file write; viewing reads the file once. Data volume for a personal diary is tiny.	Met
NFR5	Reliability & Error Handling	Handle missing files and invalid menu choices without crashing; never lose earlier data.	FileNotFoundError is caught by the view functions; invalid menu options are rejected; files are opened in append mode so earlier entries are preserved. Non-numeric quiz answers and malformed CSV rows are not yet handled (see Testing).	Partially met
NFR6	Maintainability	Code should be easy to read and extend.	One function per feature, file paths held in constants, section comments, consistent naming. The code currently lives in a single file (mindtrack.py).	Partially met
NFR7	Scalability	Remain usable as the number of entries grows over months.	Text/CSV files are adequate for thousands of personal entries. A database would be required for very large histories or search-heavy features.	Limited
NFR8	Logging / Monitoring	Record application errors for diagnosis.	Not implemented in the current version; listed under future enhancements.	Not met

4.5 System Architecture Diagram

MindTrack-CLI follows a simple four-layer structure. The user interacts only with the terminal; the menu dispatches to one of four feature modules; the modules use Python standard-library helpers; and persistent data is kept in two local files.

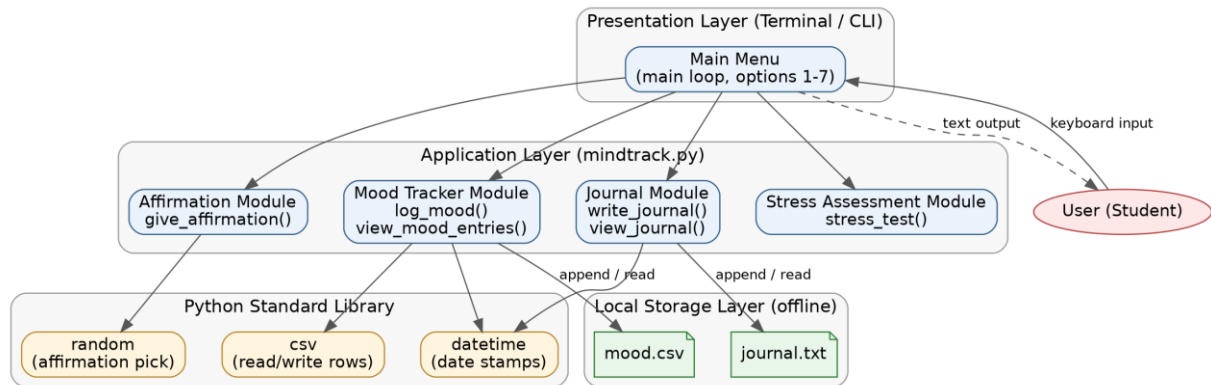


Figure 4.1: Layered system architecture of MindTrack-CLI

- **Presentation layer:** the main menu loop reads a choice with `input()` and prints results to the terminal.
- **Application layer:** Journal, Mood Tracker, Affirmation and Stress Assessment modules, each implemented as one or two functions.
- **Standard library:** `datetime` supplies the date stamp, `random` picks an affirmation, `csv` reads and writes mood rows.
- **Local storage layer:** `journal.txt` (line-based text) and `mood.csv` (comma-separated rows). Nothing is sent over a network.

4.6 Process Flow / Workflow Diagram

After start-up the program repeatedly shows the menu, reads the user's choice and performs the selected action. Every action except Exit returns to the menu, so the user can combine features in any order within one session.

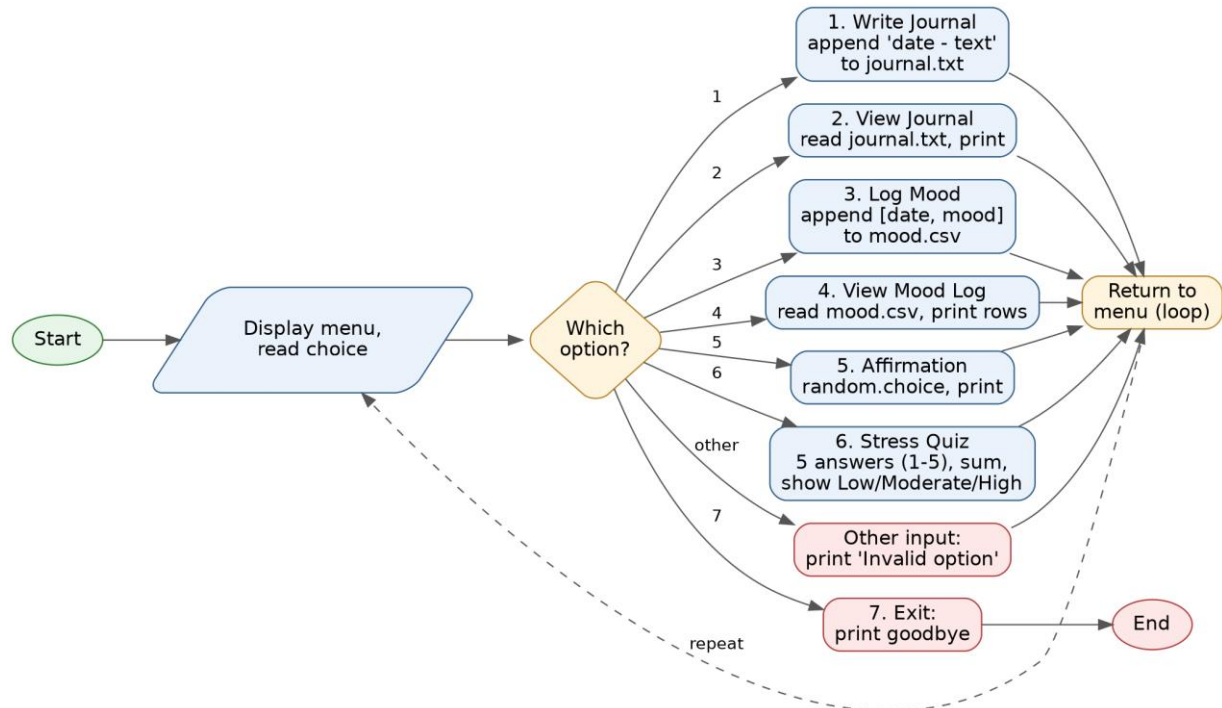


Figure 4.2: Workflow of the main menu loop and the seven options

4.7 UML Diagrams

4.7.1 Use Case Diagram

A single actor, the User (a student), interacts with seven use cases. Writing a journal entry and logging a mood both include the 'save data locally' behaviour, and taking the questionnaire includes displaying the stress level.

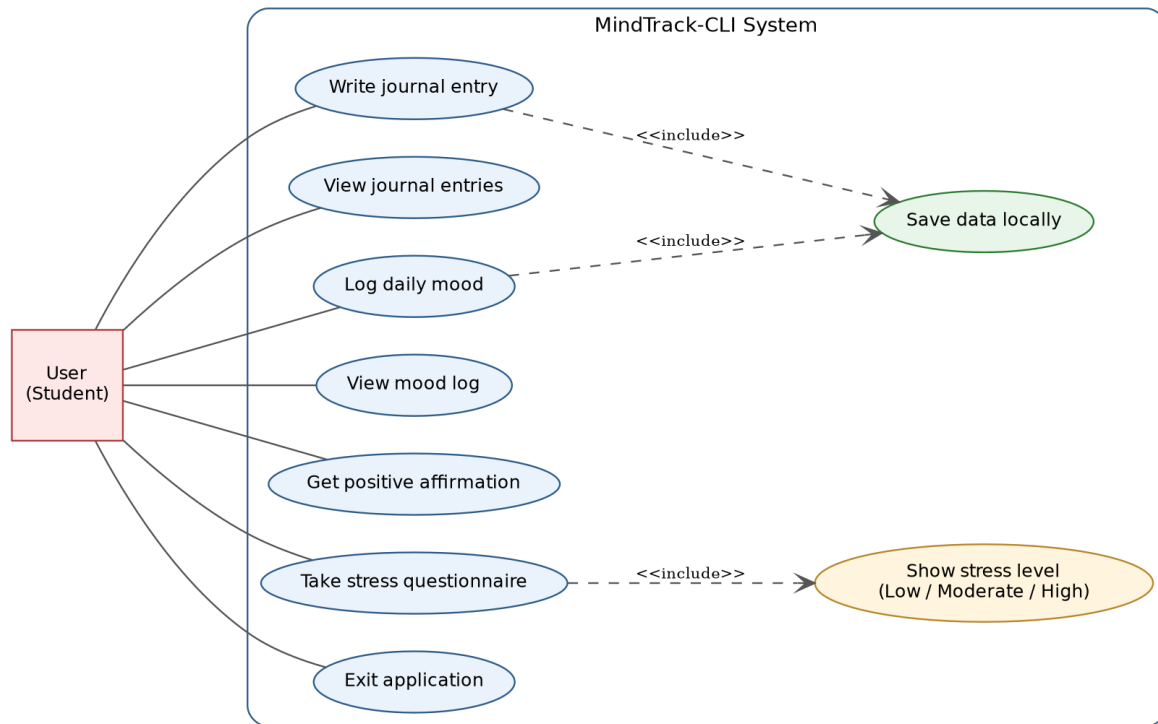


Figure 4.3: Use case diagram

ID	Use case	Main flow	Post-condition
UC1	Write journal entry	User selects 1 → system prompts for text → user types entry → system stamps today's date and appends it to journal.txt → system confirms.	Entry stored in journal.txt
UC2	View journal entries	User selects 2 → system reads journal.txt and prints all lines (or reports that no entries exist).	Entries displayed
UC3	Log daily mood	User selects 3 → system prompts for mood → user types a mood word → system lower-cases it and appends [date, mood] to mood.csv → system confirms.	Row stored in mood.csv
UC4	View mood log	User selects 4 → system reads mood.csv and prints each row as 'date → mood' (or reports that none exist).	Mood history displayed
UC5	Get positive affirmation	User selects 5 → system picks a random affirmation and prints it.	Affirmation displayed
UC6	Take stress questionnaire	User selects 6 → system asks five questions (1-5) → user answers each → system totals the score and prints Low / Moderate / High with advice.	Stress level displayed (not stored)
UC7	Exit application	User selects 7 → system prints goodbye and ends the loop.	Program terminated

4.7.2 Class / Component Diagram

The program is procedural, so the diagram shows **components** (groups of functions) rather than classes. The Main Menu component calls four feature components; the Journal and Mood components use local files, and the components depend on the standard-library modules shown.

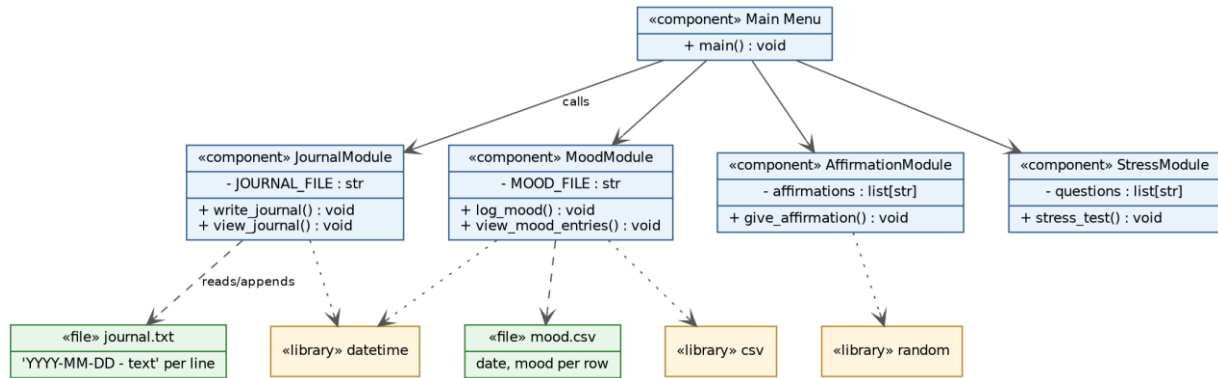


Figure 4.4: Component diagram with public operations and dependencies

4.7.3 Sequence Diagrams

Three representative scenarios are shown: writing a journal entry (file append), logging a mood (CSV append) and taking the stress questionnaire (calculation without storage).

Sequence Diagram 1: Write Journal Entry

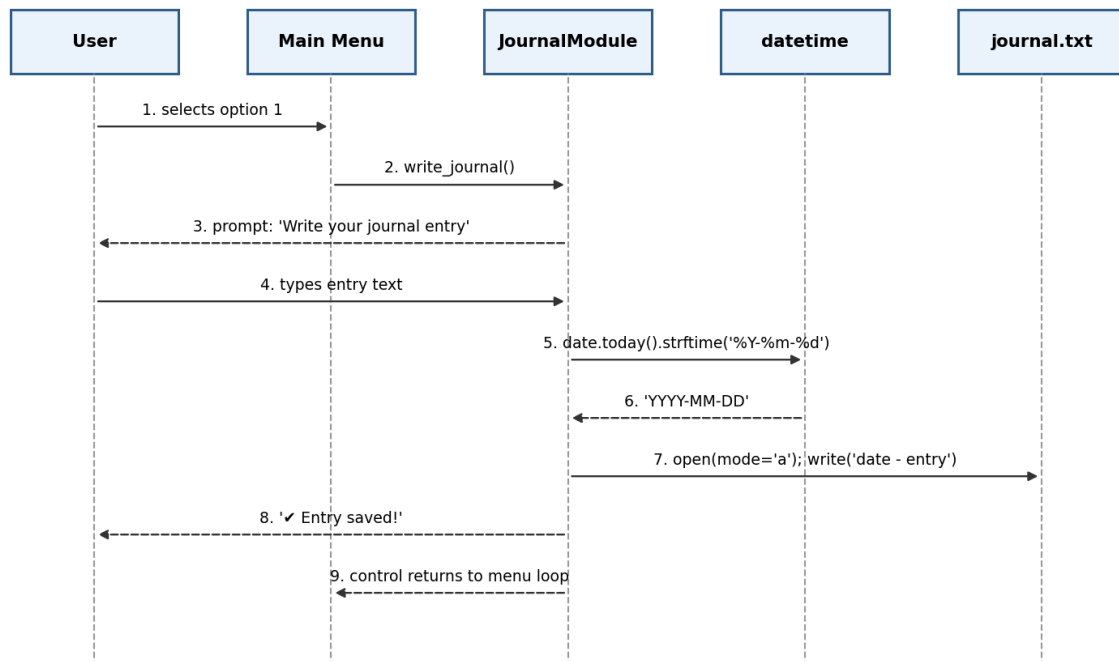


Figure 4.5: Sequence diagram - Write journal entry

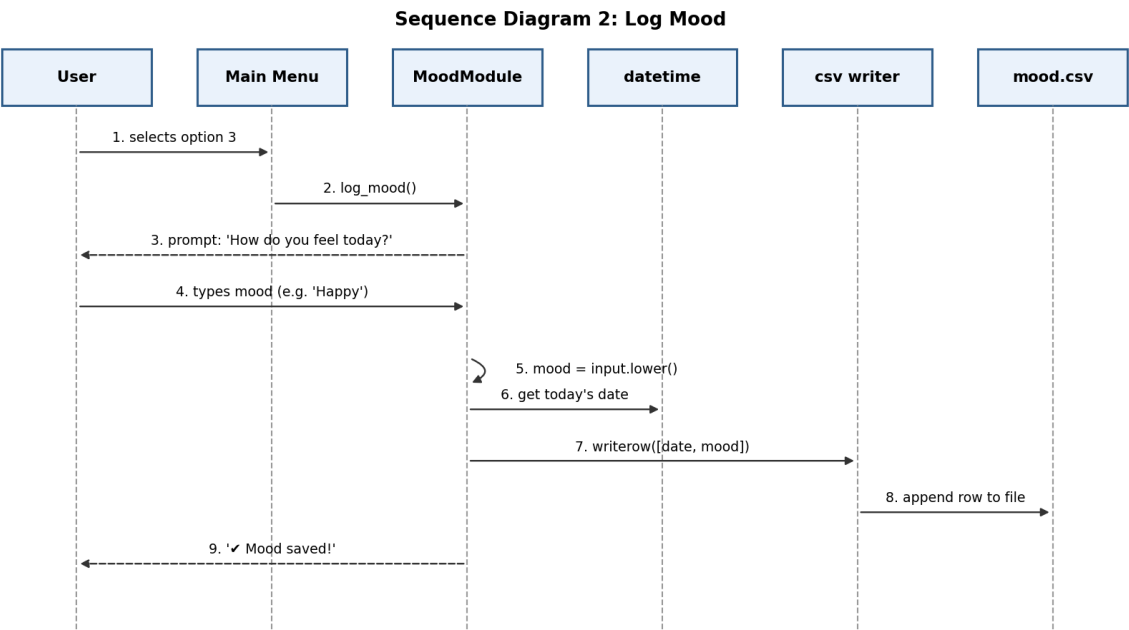


Figure 4.6: Sequence diagram - Log mood

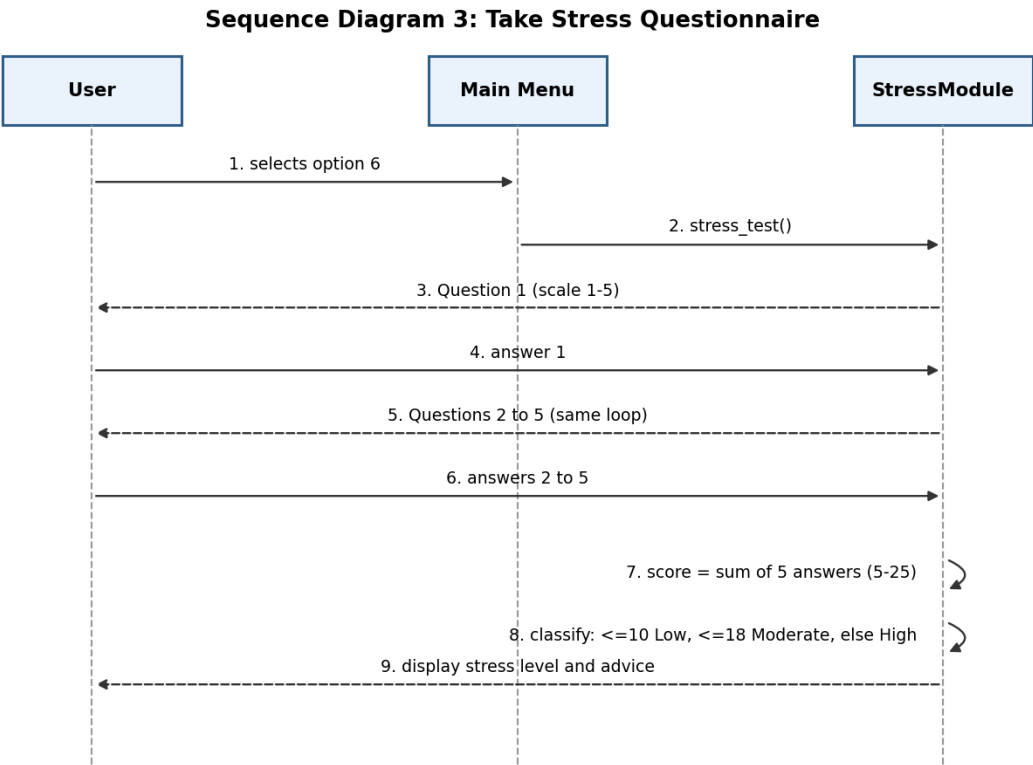


Figure 4.7: Sequence diagram - Take stress questionnaire

4.8 Database / Storage Design

Storage is file-based. This is a deliberate choice for a small offline tool: there is nothing to install, the files can be opened in any text editor or spreadsheet, and the user keeps full ownership of their data (see design rationale in the project report).

4.8.1 ER Diagram

Two entities are persisted. A logical CALENDAR_DAY entity is shown only to explain the relationship: on any date there can be zero or more journal entries and zero or more mood logs. Neither file has an explicit key column, so each record is identified by its position (line or row number).

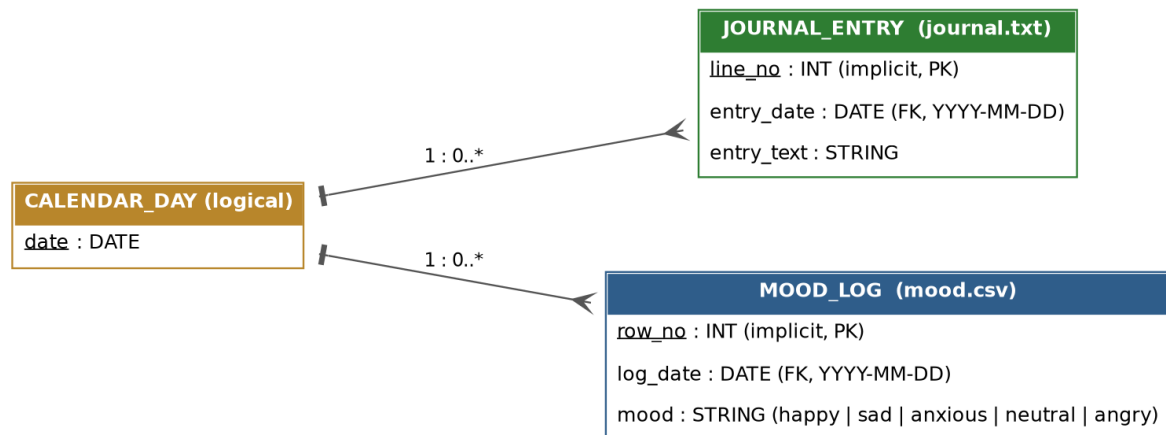


Figure 4.8: Entity-relationship diagram of the stored data

4.8.2 Schema Design

journal.txt - one line per entry

Line pattern: `YYYY-MM-DD - <entry text>`. The file is opened in append mode so earlier entries are never overwritten.

Field	Type	Format / rule	Example
date (prefix)	Text (date)	ISO format YYYY-MM-DD, taken from the system clock via <code>datetime.date.today()</code>	2025-11-23
separator	Text	Literal ' - ' (space, hyphen, space)	-
entry	Text	Single-line free text from <code>input()</code> ; no length limit	feeling happy today

```
2025-11-23 - feeling happy today
```

mood.csv - one row per log, no header row

Row pattern: `date,mood`. Written with Python's csv module (`newline=""` to avoid blank lines on Windows) in append mode.

Field	Type	Format / rule	Example
date (column 1)	Text (date)	ISO format YYYY-MM-DD from the system clock	2025-11-23
mood (column 2)	Text	Lower-case word. Prompt suggests happy / sad / anxious / neutral / angry; the value is not restricted in code.	happy

```
2025-11-23,happy
```

Storage notes

- Both files are created automatically on first write in the folder where the program is run.
- The stress questionnaire result is calculated and shown but deliberately **not stored**, which keeps sensitive results off disk.
- Files are not encrypted; users who need stronger protection should rely on device-level security (see future enhancements).

4.9 ML-specific artefacts

Section 4 of the guidelines asks for a dataset description, model selection rationale and evaluation methodology only for computation-heavy or ML-related courses. MindTrack-CLI does not use a dataset or a machine-learning model (the stress level is a fixed, rule-based score), so these artefacts are **not applicable**.

4.10 Checklist against Section 4

Required artefact	Status	Location in this document
Problem Statement	Provided	Section 4.1
Objectives	Provided	Section 4.2
Functional Requirements	Provided (9 requirements, 4 modules)	Section 4.3
Non-functional Requirements	Provided (8, minimum 4)	Section 4.4
System Architecture Diagram	Provided	Section 4.5, Figure 4.1
Process Flow / Workflow Diagram	Provided	Section 4.6, Figure 4.2
UML: Use Case Diagram	Provided	Section 4.7.1, Figure 4.3
UML: Class / Component Diagram	Provided (component form)	Section 4.7.2, Figure 4.4
UML: Sequence Diagram	Provided (three scenarios)	Section 4.7.3, Figures 4.5-4.7
Database / Storage: ER Diagram	Provided	Section 4.8.1, Figure 4.8
Database / Storage: Schema Design	Provided	Section 4.8.2
Dataset / model / evaluation (ML only)	Not applicable	Section 4.9